N

PROJECT PORTFOLIO · ZIDIO DEVELOPMENT

NexusHR

AI-Enabled Enterprise HR & Workforce Intelligence Platform

One workforce. Clearer decisions. A unified product concept for employee lifecycle, attendance, payroll, performance, and explainable workforce insights.

Prepared for
Zidio Development · Java Full-Stack Domain

Prepared by
Java Full-Stack Engineering Candidate

Portfolio edition
Version 2.0 · Industry Edition

Document date
18 October 2026

A source-grounded project overview. Performance targets are distinguished from measured results; deployment artifacts are distinguished from live operations.

Project overview

NexusHR brings daily workforce operations into one role-aware application: a React client, Spring Boot REST services, and relational persistence with a lightweight local-development profile.

4
personas represented in the demo

7
documented functional capability groups

1
integrated HR workspace

Vision and objectives

Vision

Reduce context switching across people operations by bringing employee records, attendance, leave, payroll, reviews, notifications, and workforce insights into a coherent workspace.

Product objectives

- Digitize lifecycle and approval workflows.
- Make payroll calculations and employee-facing payslips accessible.
- Support role-specific workspaces and self-service actions.
- Present workforce-risk signals with factors and suggested interventions.

Users and business value

User group	Primary use	Potential value (not yet measured)
Executive / administrator	Headcount, payroll summary, governance and security review.	Faster organization-wide visibility and oversight.
HR manager	Employee records, leave, payroll administration and workforce insights.	Less manual coordination between people processes.
Team manager	Team attendance, approvals and performance review contributions.	Clearer team-level follow-up and decision context.
Employee	Personal attendance, payslip, leave requests and review actions.	More direct access to common HR self-service tasks.

Business impact hypotheses

> **Targets, not substantiated outcomes.** Existing project materials cite reductions in administrative effort, improved retention and large-organization support. The repository does not contain a customer study, load-test report, or production availability record to validate those claims.

Functional scope 03–04

Technology and architecture 05–06

Delivery and technical practices 07–09

Roles and visual tour 10–13

Operations and reflection 14–15

Evidence basis Repository

Key features · People operations

The criteria below describe observable demo behavior. They are acceptance goals for the product, not a claim of enterprise production certification.

ID	Feature	Description	Acceptance criteria
F-01	Employee lifecycle	Employee directory, search and profile information, department context, onboarding entry point and lifecycle actions.	Authorized HR user can find employees, inspect records and start the represented onboarding flow; saved changes remain consistent in the active data source.
F-02	Attendance and leave	Simulated biometric punch terminal, attendance records, leave application and review queue.	Employee can submit a punch or leave request; reviewer can process an eligible request; status is visible in the interface.
F-03	Payroll and payslips	Gross-to-net calculation, batch payroll action, monthly payroll register and employee payslip view.	Payroll totals reconcile against configured formulas; payroll action and printable/downloadable payslip are available to permitted roles.

Illustrative payroll formula

```
Gross = Base salary + 40% HRA + 15% allowances

Net = Gross − 12% PF − 10% TDS − $150 insurance deduction
```

The formula reflects the project's documented demo rules. Tax and statutory calculations require jurisdiction-specific review before real payroll use.

Business workflow snapshots

EMPLOYEE RECORD

Find → review → update

Directory controls and employee cards make sample workforce data browsable.

LEAVE REQUEST

Apply → approve

A request has an employee, leave dates, reason, status and reviewer context.

PAYROLL RUN

Calculate → inspect

The payroll register summarizes pay-period values for demo employees.

Acceptance review checklist

Check	Expected result	Evidence needed for release
Authorization	Disallowed roles cannot invoke restricted mutation endpoints.	Controller tests and authorization integration tests.
Data integrity	Payroll totals and approval transitions remain consistent after reload.	Automated domain tests against a persistent test database.
Usability	Core employee and manager tasks complete without hidden role changes.	Keyboard and browser-based workflow testing.

Key features · Insight and communication

ID	Feature	Description	Acceptance criteria
F-04	Performance 360	Review records, weighted score dimensions, manager feedback and goal completion context.	Scorecard presents technical (40%), leadership (30%) and delivery (30%) dimensions; completed review data can be inspected.
F-05	Workforce intelligence	Rule-based attrition risk score, explanatory factors, suggested interventions and a skill-gap benchmark view.	Each result includes a risk band and rationale. Treat as a demo heuristic—not a validated predictive model.
F-06	Executive dashboard	Top-line workforce, attendance, payroll and engagement indicators with navigation to operational views.	Dashboard loads representative data and supports a user journey into relevant modules; export behavior is clearly identified.
F-07	Notifications	In-app notification drawer with unread state and workflow-oriented messages; WebSocket configuration is present in the backend.	Authorized user can view notifications and mark an item read; live delivery requires end-to-end messaging tests.

What “AI” means in this implementation

> **Explainable rules, not a trained model.** The inspected service computes risk using fixed rules for engagement, tenure, compensation and in-demand skills. A confidence value of 88.4 is assigned in source code; this is not evidence of benchmark validation or measured predictive accuracy.

Useful demo behavior

- Risk score is capped to a bounded range.
- Results are classified as low, medium or high.
- Risk factors and interventions accompany each result.
- Skill targets and current proficiency values produce gap bands.

Before operational use

- Define a lawful, representative training and evaluation dataset.
- Measure precision, recall, calibration and subgroup effects.
- Document human review and contestability.
- Review privacy, labor and employment-law obligations.

Business value hypotheses

Capability	Potential benefit	Evidence required
Employee self-service	Fewer routine HR status requests.	Baseline request volume and post-launch comparison.
Workflow visibility	More timely leave and review decisions.	Cycle-time and backlog measurements.
Workforce indicators	Earlier, more structured manager conversations.	Human-reviewed intervention study; no automated employment decisions.

Technology stack

The repository separates the React user interface from the Spring Boot API and supports a lightweight developer profile alongside container-oriented deployment configuration.

Category	Technology	Rationale in this project	Alternative / note
Language / runtime	Java 21 target; Maven 3.9+	Typed backend services and a standard Maven build. CI config selects Temurin 21.	Java 24 was also used locally; supported runtime should be pinned to CI target.
Backend framework	Spring Boot 3.3.4	REST controllers, dependency injection, persistence and service composition.	Micronaut or Quarkus are alternatives for different runtime/startup constraints.
Security	Spring Security 6; JJWT	Stateless bearer-token filter, BCrypt encoder and method-level role checks.	External OIDC provider recommended for production identity lifecycle.
Data access	Spring Data JPA / Hibernate 6	Repository abstractions map HR domain entities to relational storage.	JDBC/jOOQ can suit explicit SQL and reporting-heavy query paths.
Local database	H2 (dev profile)	Fast setup for evaluation and local development.	PostgreSQL is configured for production profile; validate schema behavior across both.
Production database	PostgreSQL 17 (Docker Compose)	Relational persistence with transactions and durable volume in Compose.	Managed PostgreSQL preferred for backups, failover and operational support.
Frontend	React 19, TypeScript, Vite	Component UI with static build and development server.	Next.js or Remix could add server rendering and route-level data loading.
Styling / icons	Tailwind CSS v4; Lucide React	Utility-first styling and consistent icon set already used by the UI.	Design-system tokens and component library can increase consistency.
Containers	Docker; Docker Compose	Backend, frontend, PostgreSQL and Redis service definitions are provided.	Kubernetes manifests are a separate deployment option, not proof of a running cluster.
Operations	GitHub Actions; Prometheus config	Build/test workflow and scrape targets are checked into the project.	Metrics exporters, alert rules, dashboards and deployment stages need verification.

> **Configuration audit note.** Redis is provisioned in Compose, but the inspected application configuration does not establish a Redis-backed cache/session path. The Prometheus file lists PostgreSQL and Redis exporter targets; exporter services are not defined in the reviewed Compose file.

Build and delivery commands

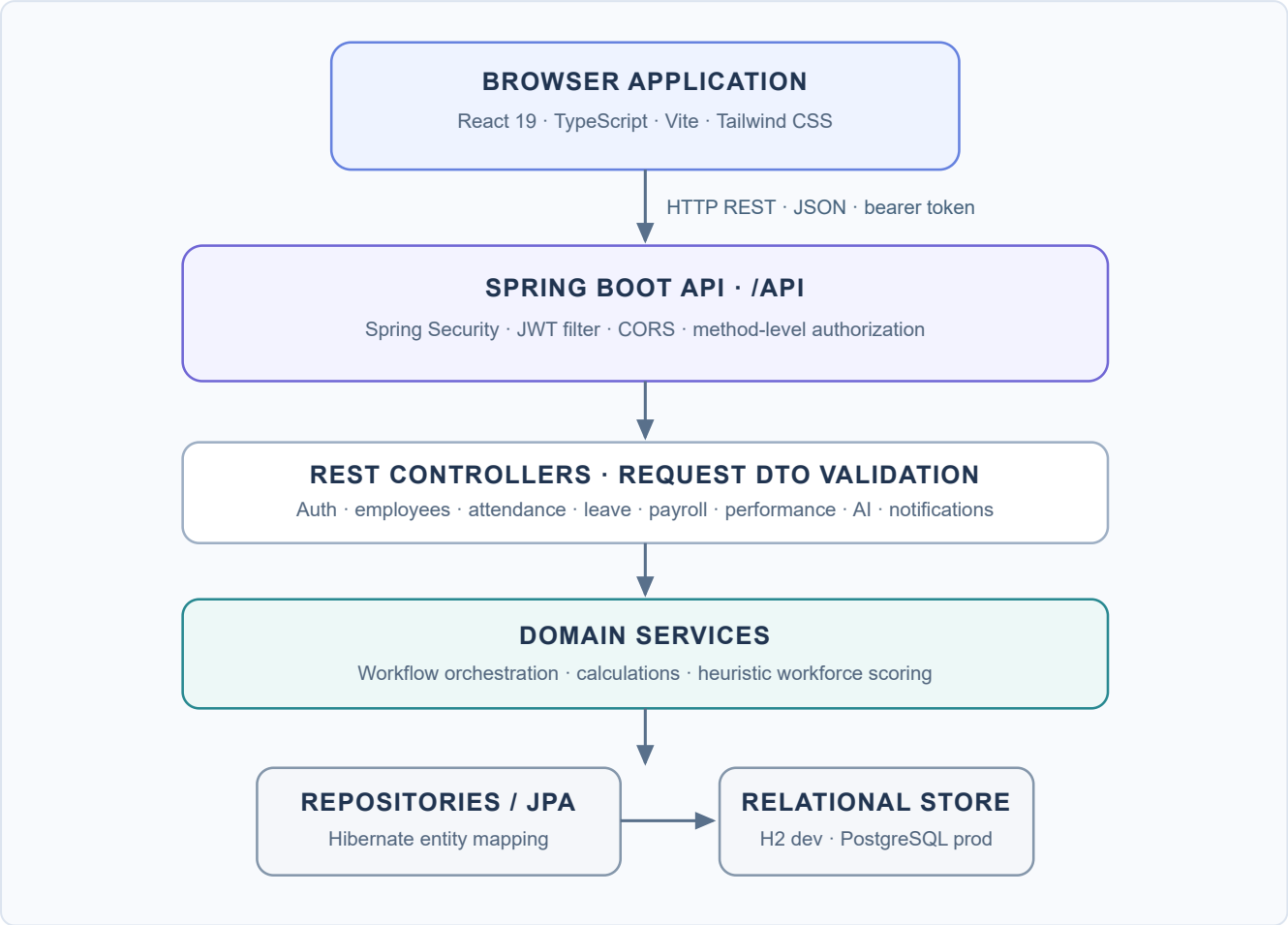
```
Backend: cd backend && mvn clean test

Frontend: cd frontend && npm ci && npm run build

Local stack: docker compose up --build
```

Architecture at a glance

A browser client calls a single Spring Boot API surface. Controllers delegate to domain services and repositories; the active profile selects the database configuration.



Request path

01 · AUTHENTICATE
Login endpoint

Credential check returns user identity and token data; the web client stores session details locally.

02 · AUTHORIZE
Security boundary

A JWT filter populates the security context; controller method annotations apply role rules.

03 · PERSIST
Domain service

Typed repositories and JPA entities implement persistence and business workflows.

Architecture boundaries to preserve

- Keep role decisions server-enforced; client-side menu filtering is not a security boundary.
- Keep the demo heuristic separate from any future trained-model inference service.
- Externalize secrets and environment-specific origins before shared or public deployment.

Execution timeline

A four-week delivery outline adapted from the project's existing documentation. It is a proposed/reconstructed plan; commit history was not available to validate actual dates or durations.

Week 1 · Foundation

DAYS 1–7

- Confirm HR use cases and role boundaries.
- Bootstrap Spring Boot and Maven.
- Model users, employees and departments.
- Implement authentication and persistence.

Checkpoint · login + entities

Week 2 · Workflows

DAYS 8–14

- Build React shell and navigation.
- Implement employee and attendance views.
- Add leave requests and approval flows.
- Deliver payroll and review screens.

Checkpoint · core workflows

Week 3 · Insights

DAYS 15–21

- Add dashboard metrics and notifications.
- Implement explainable risk rules.
- Add skills benchmark comparison.
- Review role-specific navigation.

Checkpoint · integrated demo

Week 4 · Release

DAYS 22–28

- Containerize backend and frontend.
- Exercise CI tests and production build.
- Review Kubernetes and monitoring config.
- Capture visuals and complete documentation.

Checkpoint · portfolio release

Milestones and exit criteria

Milestone	Exit criteria	Evidence to retain
M1 · Foundation	Developer can start the app; login and seeded sample records are available.	Setup guide, startup check, auth tests.
M2 · Workflows	Employee, attendance, leave and payroll tasks work through the UI and API.	Workflow tests and API examples.
M3 · Insights	Dashboard, rule explanations and notification states are visible.	Known-input tests and screenshot evidence.
M4 · Release	CI checks pass; deployment artifacts are reviewed; limitations are stated.	Build logs, container scan, runbook, release checklist.

Checkpoints to add to a real plan

Risk / privacy review

Before loading employee data: approve data inventory, retention, access, encryption, incident response and lawful-use basis.

Go-live readiness

Before production: test recovery, database migration, concurrency, accessibility, audit events and rollback procedure.

Technical highlights · Security

The implementation contains foundational controls, but the project should be treated as an evaluation/demo application until a security review and deployment hardening are complete.

Control area	Repository evidence	Practical assessment
Identity	Spring Security authentication manager; BCrypt password encoder; JWT access and refresh token paths.	Present in code. Rotate secrets and add key management, token revocation/rotation policy and abuse controls.
Authorization	Method security enabled; controllers use `@PreAuthorize` for selected actions.	Promising server-side boundary; audit every controller and every object-level ownership check.
Input handling	Request DTOs and Jakarta validation are used on selected controller inputs; JPA repositories structure data access.	Validate every endpoint and enforce business invariants at the service layer; test malformed and oversized inputs.
Transport / CORS	CORS bean accepts wildcard origin patterns with credentials; application YAML also declares an origin list.	Configuration mismatch / broad policy risk. Use a single explicit allowlist and verify effective browser policy.
CSRF / sessions	CSRF is disabled and Spring sessions are stateless.	Appropriate only for a correctly managed bearer-token API; protect browser token storage and review XSS exposure.
Secrets	JWT secret appears in application YAML; Compose includes a sample database password. K8s references a Secret for DB credentials.	Do not reuse committed values. Rotate, remove from source control, and inject through a secret manager.
Abuse controls	No rate-limiting implementation was identified in reviewed configuration.	Add login throttling, request limits, account lockout policy and monitoring before exposure.
Logging / audit	Security/audit UI and Prometheus scrape configuration exist in the project.	Verify immutable event capture, sensitive-data redaction, retention and alert response. UI labels alone are not audit evidence.

OWASP-oriented threat checklist

Access and injection

- Test horizontal and vertical privilege boundaries.
- Verify resource ownership on reads and updates.
- Prefer parameterized persistence; test validation and error paths.
- Do not rely on hidden navigation to authorize API access.

Secrets and availability

- Rotate demo and signing credentials before deployment.
- Limit CORS to reviewed client origins.
- Add login rate limits and security event alerts.
- Use TLS, backups, restore drills and dependency scanning.

> **Public-demo warning.** Demo credentials, H2 Console settings and any checked-in signing keys must be treated as unsafe for a public or shared deployment. Replace and rotate them first.

Performance and scalability

The materials specify ambitious service objectives and include horizontal-scaling configuration. No reproducible load-test results, measured latency distributions, or production uptime evidence were found in the repository.

Measure	Documented target	Verified evidence	Next validation
Core API latency	< 300 ms	Target only	Publish environment, dataset, request mix, p50/p95/p99, error rate and run duration.
Dashboard load	< 2 s	Target only	Measure cold/warm load on agreed device, network, browser and data volume.
Concurrent users	10,000+ cited in report	Not load-tested	Run staged load tests; inspect database pools, query plans, CPU and memory.
Availability	99.95% cited in report	Not measured	Define SLO window, deploy multi-zone dependencies, and collect uptime history.
Attrition accuracy	Above 80% cited in report	Not validated	Independent, leakage-safe evaluation on representative labeled outcomes.

Scaling provisions present in configuration

KUBERNETES
3 backend replicas

Deployment requests 250m CPU / 512 MiB and caps at 1 CPU / 1 GiB.

HORIZONTAL AUTOSCALING
2–10 replicas

HPA targets average 70% CPU and 80% memory utilization.

PROBES
Readiness / liveness

Manifest calls `/api/public/health`; validate endpoint response and startup timings.

Performance improvement plan



The bars indicate recommended sequence only; they are not measured performance or completion percentages.

Likely bottlenecks to investigate

- Attrition analysis loads the employee collection and evaluates each employee in application memory.
- Dashboard and directory endpoints should be profiled for N+1 entity loading and pagination needs.
- Large payroll batches need transaction sizing, idempotency, locking and failure-recovery tests.
- Horizontal replicas require stateless operation, shared durable storage and consistent signing keys.

Role-based access and user journeys

The sign-in experience offers four account choices. A successful authentication should determine the account role; switching personas requires signing out and signing in with a different account.

Role	Representative journey	UI navigation available	Server checks
Executive Admin	Review workforce overview → inspect payroll → audit controls.	Dashboard, employees, attendance, leave, payroll, performance, AI, security.	Controller-level authorization annotations apply to selected operations.
HR Manager	Find employee → review leave → check pay and retention indicators.	HR operational modules and governance views.	Role authority checked by backend where configured.
Team Manager	Review team attendance → approve leave → contribute performance feedback.	Team-oriented modules; some admin-oriented views are hidden.	Object ownership / team scoping should be tested for every endpoint.
Employee	Check personal time → submit leave → view payslip and review.	Self-service modules and personal actions.	Self-only record access must be enforced server-side.

Authentication path



Sign out clears browser session data and returns to the login screen.

Demo account handling

> **Evaluation accounts only.** Accounts and passwords documented by the project are hard-coded sample credentials in demo setup. Do not publish or reuse them on a publicly reachable deployment; replace seeded passwords and rotate signing/database secrets.

When the API is unreachable, the frontend contains a demo-account fallback. That fallback is for local evaluation and is not an identity provider. A production release should fail closed, use a controlled identity service, and avoid storing bearer credentials in browser local storage.

Visual 01 · Sign-in experience

Captured from the running NexusHR frontend on 18 October 2026. This is a real application screenshot, not a design mockup.

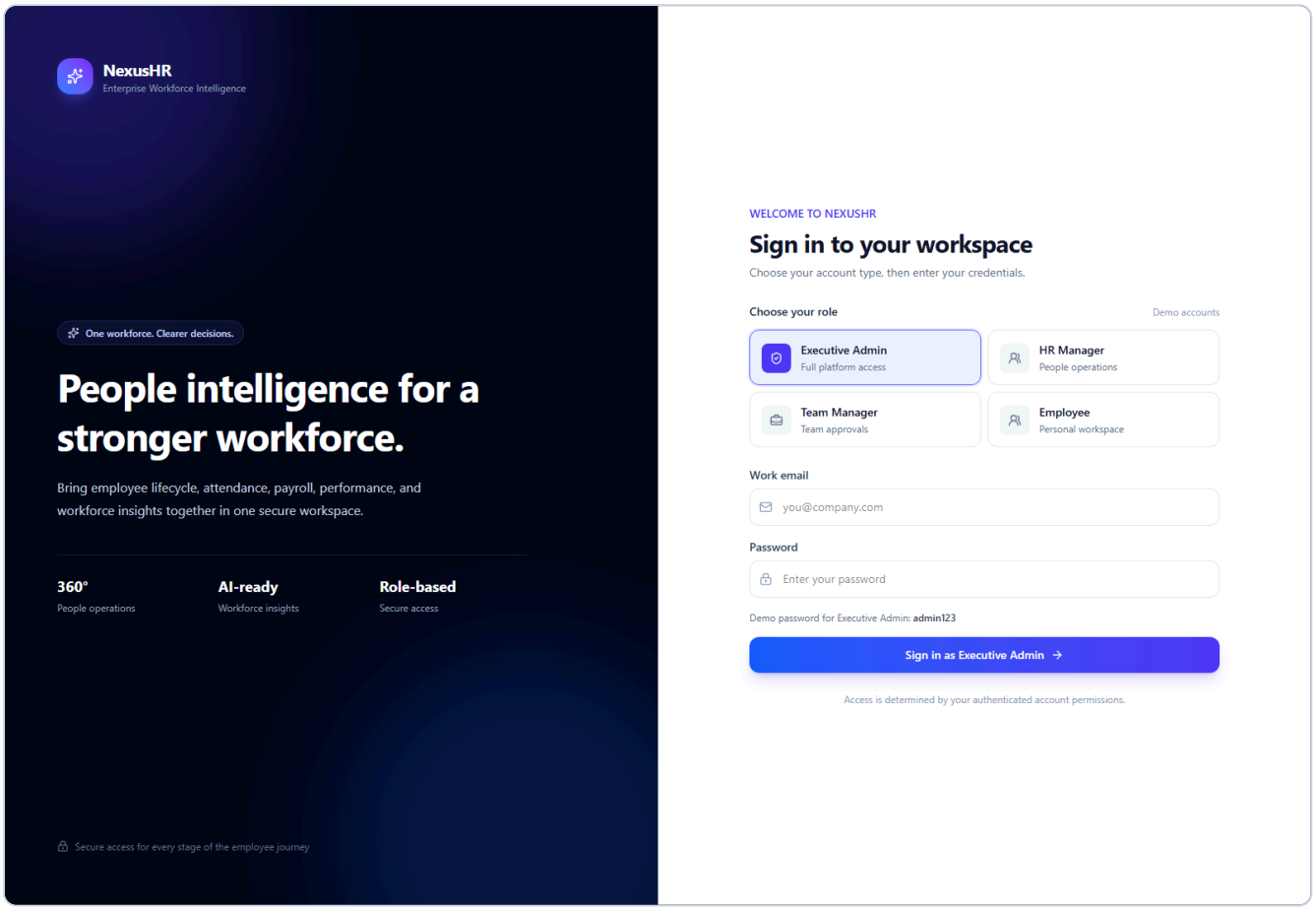


Figure 1. Role selection and email/password login. The demo UI identifies sample credentials and routes users to their authenticated persona.

Visual observations

Choice before credentials

Four role cards establish the intended access context before login.

Desktop / responsive

The split hero collapses to a compact brand header at smaller widths.

Demo disclosure

The screen labels role accounts as demo accounts and shows the selected demo password.

> **Portfolio capture note:** this screenshot was rendered from the app running locally. It does not represent a deployed public service or production user data.

Visual 02 · Workforce dashboard

The dashboard screenshot shows the role-aware navigation, workforce KPI cards, a simulated punch terminal, and rule-based attrition insights.

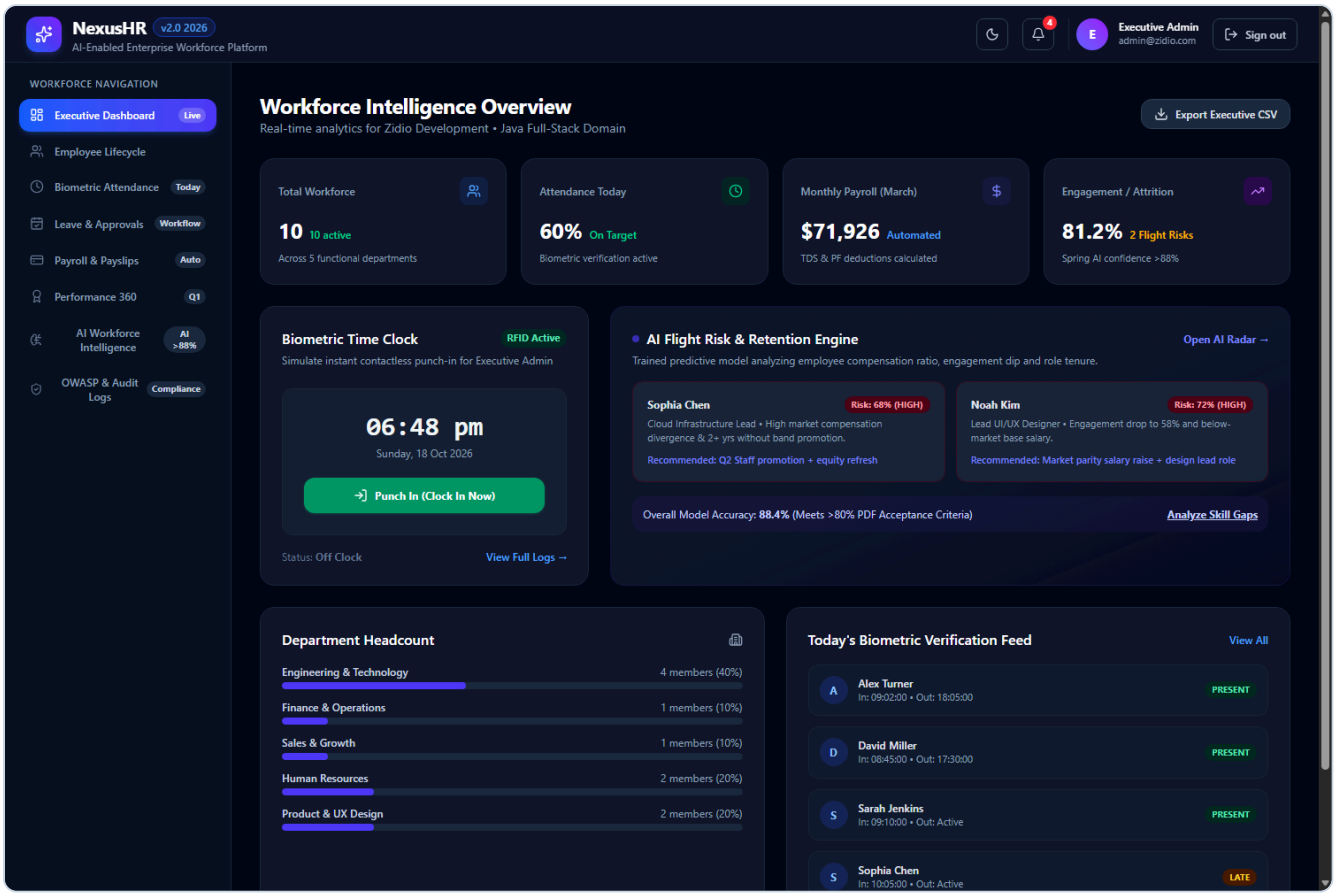


Figure 2. Admin dashboard captured from the running application using seeded/demo data. Values are illustrative application data and not operational metrics.

How to read the dashboard

Summary cards

Headcount, attendance, payroll and engagement/attrition data summarize sample records for the selected period.

Action modules

The lower dashboard exposes common tasks and navigation into attendance, AI analysis and department views.

> **Interpretation.** "Real-time" and "AI confidence" labels in the demo UI describe the intended feature experience; they should not be read as service-level or model-validation evidence.

Visual 03 · Employee directory

The employee directory is a browsable view of the seeded workforce, with search, department filtering and employee cards.

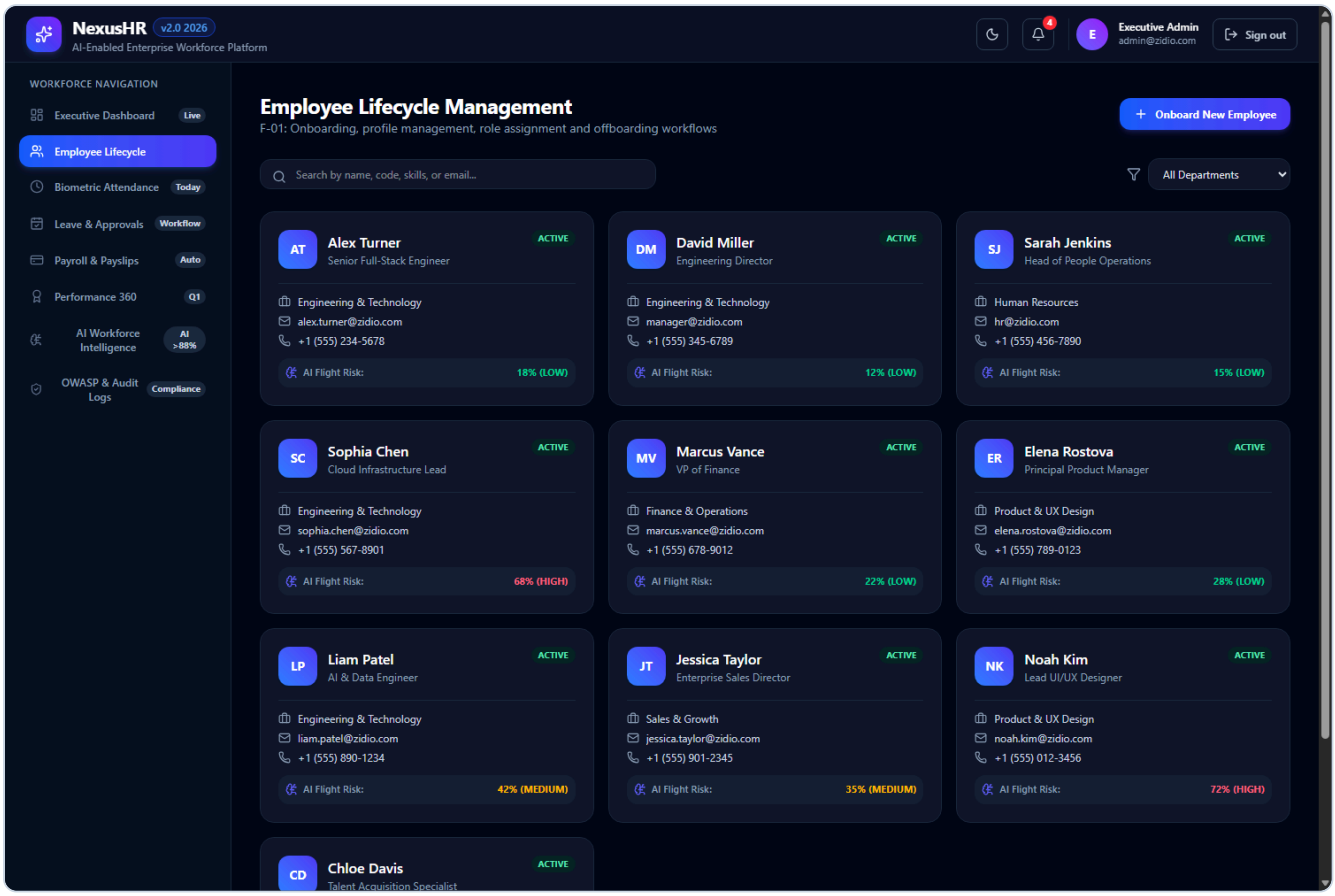


Figure 3. Employee directory. The capture displays demo records and should not be populated with real employee data until privacy controls are reviewed.

Useful next step

Add pagination, explicit empty/error states and field-level access control before connecting larger datasets.

Privacy requirement

Minimize personal details in search results; log sensitive record access and define retention rules.

Visual 04–05 · Payroll and AI insights

Two complementary views: payroll arithmetic and a rule-explanation interface for workforce risk review.

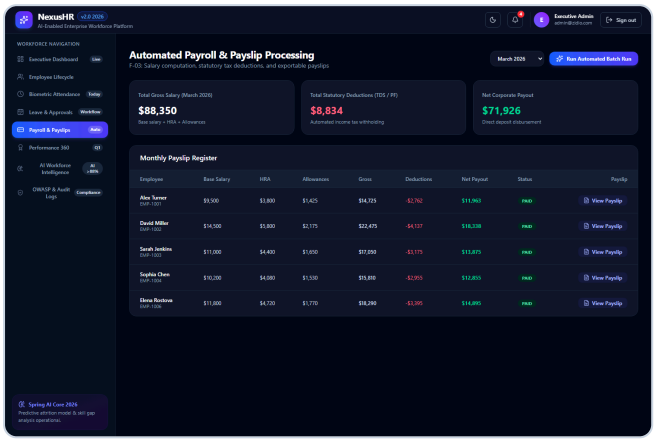


Figure 4. Payroll register · seeded pay period.

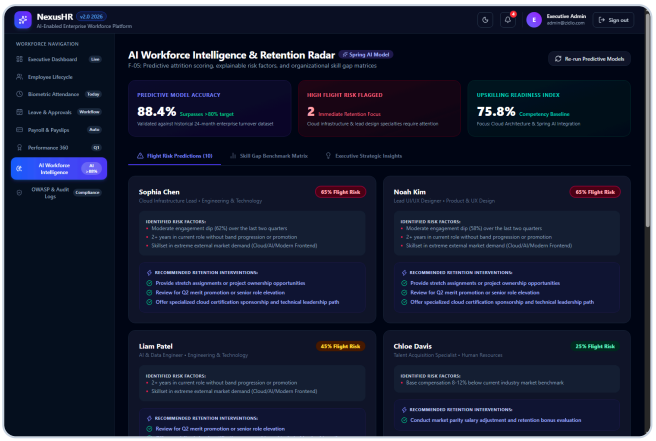


Figure 5. Workforce insights · rule-based demonstration.

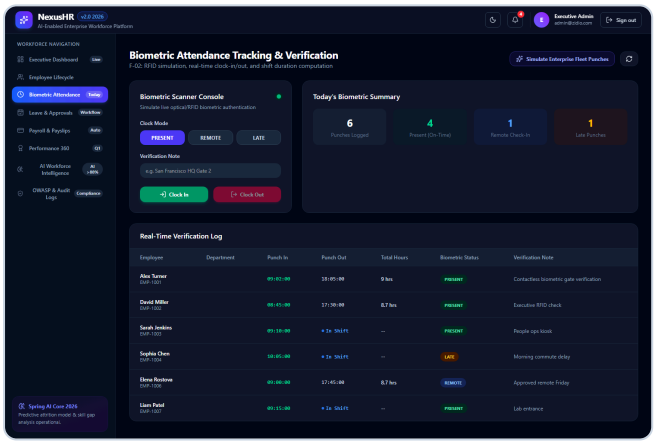


Figure 6. Attendance console · simulated punches and sample log.

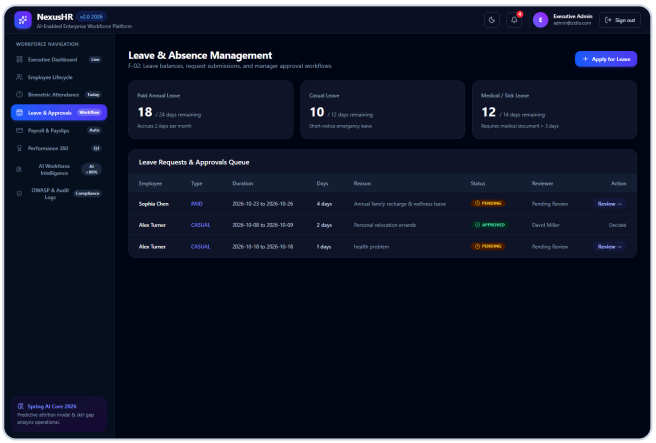


Figure 7. Leave workflow · sample balances and approval queue.

Interpretation and safeguards

Payroll is a demonstration

Validate local tax, benefits, rounding, currency, approvals and audit requirements before handling actual compensation.

Risk scores need governance

Human review, bias evaluation, model documentation and employee recourse are prerequisites for any consequential use.

> **Important:** application-rendered values and labels are sample outputs. Do not present the shown model accuracy, payroll figures or retention flags as independently audited facts.

Deployment, verification and roadmap

NexusHR includes local, container and Kubernetes deployment artifacts. The reviewed repository does not identify a cloud host or prove a live production deployment.

Deployment and operations

Area	Present in repository	Operational status / follow-up
Local development	Spring Boot + H2 and Vite development server.	Suitable for demo; use environment-aware ports and proxy config.
Docker Compose	Frontend, backend, PostgreSQL 17 and Redis 7 services.	Configuration only. Validate image builds, secrets, readiness and persistent recovery.
Kubernetes	Backend deployment/service, HPA and frontend deployment manifests.	Not evidence of a deployed cluster. Verify ingress, secrets, migrations and rollback.
CI/CD	GitHub Actions runs Maven tests, frontend build and Docker build on main.	No registry publish or deployment step was seen in the reviewed workflow.
Monitoring	Prometheus scrape configuration names app, PostgreSQL and Redis targets.	Confirm endpoint exposure, exporter services, dashboards, alerts and runbooks.
Health checks	Kubernetes readiness/liveness probes call `/api/public/health`.	Confirm status semantics, timeout behavior and dependencies before rollout.

Validation performed for this portfolio

- Frontend production build and TypeScript compilation completed successfully during the login-page work.
- Browser walkthrough confirmed role sign-in and sign-out returning to the login page.
- Dashboard, employee directory, payroll, and AI insights screenshots were captured from the running local application.
- No load test, security penetration test or production deployment was performed for this document.

Reflection and next steps

Engineering learnings

- End-to-end role flows are easier to reason about when demo persona switching is replaced with authentication.
- Targets should be documented separately from measured outcomes.
- Security posture depends on effective configuration, not only framework defaults.

Roadmap

1. External identity, key management, strict CORS and throttling.
2. Persistent test database, API authorization tests and CI security scanning.
3. Payroll jurisdiction review, audit retention and accessible UI tests.
4. Reproducible performance plan; model evaluation and human governance.
5. Production runbook, recovery test, alerting and real deployment evidence.

Closing note. NexusHR is a broad, visually demonstrable HR platform concept with a clear path from local prototype to a production-ready system—provided that identity, privacy, correctness, performance and operational controls are validated before real employee use.

Source basis: project README and docs; frontend package, authentication, API and UI modules; backend Spring Security, authentication and AI service; Docker Compose; GitHub Actions; Kubernetes and Prometheus configuration. Reviewed 18 October 2026.